CS781 Capstone Project: In-House Legal Intelligence System

Yash Yadav
261110076
yashy26iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This report describes an in-house legal NLP stack built from a single 110M-parameter transformer pretrained from scratch on 3.5B tokens of Indian legal text and adapted to six downstream tasks: named-entity recognition, rhetorical-role labelling, judgment prediction, statute identification, abstractive summarization and prior-case retrieval. Four Phase-1 architectures were trained at matched token budget; a prefix-LM scored best on generation. Downstream, parameter-efficient adaptation (LoRMA at 0.27% of weights) stayed within noise of full fine-tuning on every task, and a half-depth distilled student matched or beat its teacher three times. What did move scores was choosing which 1024 tokens the model reads, worth 0.189 macro-F1 on judgment prediction, and the rule converting model output into predictions, worth +0.139 on statute identification. Eight pre-registered predictions were refuted, two of them visible only under matched decoding.

1 Competition Result

Codabench username: y_261110076 **Roll number:** 261110076
Code: `https://github.com/0nibaba/CS781-Capstone`

Table 1: Official phase scores and ranks. Ranks recorded.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM pretraining	BERTScore F1 **0.8343**	46	5
2	Legal NER	Strict macro-F1 **0.5683**	33	6
3	Rhetorical role prediction	Macro-F1 **0.5023**	60	6
4	Court judgment prediction	Macro-F1 **0.6997**	77	6
5	Legal statute identification	Macro-F1 **0.3978**	37	7
6	Abstractive summarization	BERTScore F1 **0.8260**	102	4
7	Prior case retrieval	Micro-F1@7 **0.2965**	103	5

Total valid Codabench submissions across all phases: 39. Every phase met the minimum of four distinct submissions and Phases 2–5 exceeded it; no submission repeats an earlier one, each differing in architecture, adaptation technique, input representation or decision rule.

2 Introduction and Problem Description

A law firm that cannot send client documents to a third-party API still needs what such APIs provide. That constraint, confidentiality, not capability, is what makes a small on-premises model worth building. Everything here comes from one foundation model trained from scratch on public Indian legal text, under a 50–300M parameter cap, with no pretrained language model, no external legal corpus and no external LLM at inference.

Phase 1 pretrains the language model, scored on continuing a 512-word prompt with 128 words under BERTScore F1. The six downstream phases take a judgment as input and produce character-offset spans for twelve entity types under strict macro-F1, where a one-character error scores zero (Phase 2); a rhetorical role for each sentence across two legal domains (Phase 3); a binary appeal outcome over balanced classes (Phase 4); a subset of 100 statutes (Phase 5); an abstractive summary (Phase 6); and a ranking of exactly

seven prior cases under micro-F1@7 (Phase 7). Each required at least four substantially different adaptation experiments, LoRMA and knowledge distillation compulsory, every one initialised from a Phase-1 checkpoint.

Three questions ran through the project. How much of a task is decided by *which* part of a long document the model sees, given that 1024 tokens covers roughly a fifth of a typical judgment? Is parameter-efficient adaptation competitive with full fine-tuning when labelled data ranges from 70 documents to 43,000? And, the question that mattered most, how much of an apparent modelling result is really a property of the decision rule that turns scores into predictions?

I built the tokenizer, the tokenized corpus, four Phase-1 architectures with their own training and generation loops, and heads, pipelines, adaptation methods, evaluation harnesses and submission builders for six downstream tasks. I did not attempt hierarchical encoders exceeding the 1024-token position table (Sections 4.5, 4.7) or listwise ranking objectives.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

All data came from the official `Exploration-Lab/CS-781-Capstone` configurations. Phase 1 draws on High Court judgments, Tribunal orders, Supreme Court judgments and Acts; Table 2 gives the sizes and the structural fact that drove each phase. Three splits needed constructing: Phase 2 ships two folds of 35 documents and no development set, so I cross-validated across folds; Phase 6 ships no public split, so I held out 500 training documents under a fixed seed; and Phase 7 gives each split its own candidate pool, which I never mixed.

Table 2: Dataset sizes and the constraint that drove each phase's design.

Phase	Train	Eval	Structural constraint
1	3.51B tokens	1000 prompts	median document shorter than context
2	70 docs (2 folds)	35 docs	100% of documents exceed 1024 tokens
3	80 docs	10 dev docs	dev lacks 2 of 13 labels
4	37,387 cases	1,988 / 1,517	prior shifts 59/41 $\rightarrow$ 50/50
5	42,750 docs	10,181 / 13,019	100 statutes, 3.9 labels/doc
6	7,030 pairs	500 / 100	summary alone fills the window
7	827 queries, 4,320 cands	118 / 1,023	7 predictions, 8.25 gold

3.2 Corpus statistics that informed design

After deduplication Phase 1 holds Tribunal 1.62B tokens (46.1%), High Courts 1.50B (42.7%), Supreme Court 253M (7.2%) and Acts 142M (4.0%). The two sources the model handles worst, Supreme Court (validation perplexity 11.5) and Acts (8.24), are the two barely represented and the two the test distribution emphasises, so 88.8% of compute goes to the two easiest distributions. Rebalancing is bounded rather than free: Acts holds only 142M tokens, and reaching 20% of a 3B-token run needs 4.2 epochs over it, so the ceiling is set by repetition risk rather than preference.

I measured document-separator density directly, 2000 random 1024-token windows per source, and it corrected my own premise. The raw median High Courts document is 282 words, implying five or six documents per block, but sub-200-word documents had been routed out of the mixture, so the real figure is 1.99 per window (Table 3): milder than the Phase-1 ablation assumed, which I state rather than quietly drop.

Table 3: Two independent corpus properties, measured per source, and the document-masking penalty they jointly predict (Section 5.2).

Source	Documents per 1024-token window	Adjacent pairs > random p99	Masking penalty
tribunal	1.49	**11.3%**	+0.0466
acts	1.12	5.5%	+0.0022
supreme	1.22	2.1%	−0.0004
high	**1.99**	1.4%	+0.0008

The more consequential finding is near-duplication. Exact MD5 deduplication removed 123,603 documents, yet 11.3% of *adjacent* Tribunal pairs exceed the 99th percentile of random-pair similarity, the closest sharing parties, dates and a Jaccard score of 1.000, the same matter filed twice. This one property explains the Phase-1 ablation and separately contaminated my held-out set (Section 5.1).

Downstream, four facts set the agenda. Phase 2's twelve entity types span an 8.6× frequency range, so under macro-F1 R.COUNSEL (184 examples) counts as much as DATE (1,575), and WIT, APP and RESP are all person names separated only by narrative role. Phase 3's self-transition rates of 94% and 75% make it segment labelling, with a copy-previous-gold oracle at 0.754. Phase 4's training prior is 59/41 against balanced evaluation, and Phase 6's references share 86.1% of their token types with the source, which makes extraction a serious competitor on an embedding metric.

3.3 Preprocessing decisions

I trained a byte-level BPE tokenizer on a 350MB sample reweighted toward Acts and Supreme Court text, comparing four vocabulary sizes with and without digit splitting. At 32k, fertility is 1.296 against GPT-2's 1.353; 48k buys a further 1.3% for 12.3M extra embedding parameters, a poor trade against a fixed budget. Explicit digit splitting *cost* 7.6% fertility, and a token dump showed why, byte-level BPE already splits digits. The corpus was tokenized into `uint16` memory maps with `<|endoftext|>` after every document, sub-200-word documents written separately so their inclusion is a runtime switch rather than a re-tokenization.

Phase 2's strict metric makes tokenizer alignment a hard ceiling, so I measured it before training: offsets → token indices → offsets over all 8,393 gold spans. Requiring token-boundary alignment recovers 3.66%, because byte-level BPE attaches the leading space to the word. Expanding to every token a span touches and stripping whitespace, the `trim` convention, recovers 98.18%, a macro-F1 ceiling of 98.66%. Stripping trailing punctuation looked free, since every sampled failure had the shape "start correct, end late". It is not: 397 gold spans legitimately end in punctuation, 217 of them citations where the bracket belongs to the entity, and the variant scored 3.6 points worse. I counted those spans before running the comparison and the count predicted the outcome.

Every Phase-2 document exceeds the window (median 5,542 tokens), so I used stride-512 windows with predict-the-middle reconstruction; Phases 4–7 each required choosing *which* window, and that choice dominates everything else (Section 4.5).

4 Approach and Model Evolution

4.1 Phase-1 foundation model

The decoder is a 12-layer, 12-head, d_{model}=768 pre-norm transformer with RMSNorm in fp32, GELU, learned absolute positions, no biases, tied input and output embeddings, and residual projections scaled by $1/\sqrt{2n_{\text{layer}}}$: 110,316,288 parameters by direct count, inside the 50–300M bound. Tying is the sizing decision worth defending. It makes prediction a dot product between the final hidden state and each token's embedding, so the word going in and the word coming out occupy the same point, and it frees 24.6M parameters, roughly two further layers.

I verified the pieces before any long run rather than assuming them: manual and FlashAttention paths agree to 1.5×10^{-7}; perturbing position 10 leaves positions 0–9 bit-identical, and perturbing padded positions leaves real positions bit-identical; initial loss is 10.44 against $\ln 32000 = 10.37$; and a 400-step overfit test reaches 0.0000 with verbatim reconstruction including references such as "173(2)".

At 66.5k tokens/s on an A40 a full 3.5B-token run costs 14.6 hours, so I trained four architectures at a matched 1.494B tokens: the plain decoder; a prefix-LM, bidirectional over a prompt sampled uniformly from $[0.7T, 0.9T]$ and causal over the continuation; a windowed local–global variant; and a 4-encoder/6-decoder model trained with span corruption. Generation uses my own sampling loop rather than a library, since the architectures are custom and sampling behaviour is part of what Phase 1 demonstrates.

Two things failed early. The first generation loop had no KV cache, making a 1000-prompt submission a 30-minute job. More interestingly, the encoder–decoder cannot reach the required output length under standard span corruption: a 15% budget over 1024 tokens is 150 masked tokens across about six spans, so no span approaches the ∼166 tokens Phase 1 needs (p99 length 78). I added a long-terminal mode, 15% of

examples carrying one 120–200 token span at the sequence end, before training rather than discovering it after eight GPU hours.

4.2 Transfer protocol

Every downstream model initialises from a Phase-1 checkpoint; nothing trains from scratch and no external weights are used. Each phase attaches its own head to the shared backbone: token classification with an optional linear-chain CRF (Phase 2), separator-token "marker" pooling (Phase 3), pooled document classification (Phases 4–5), encoder–decoder or prompt-conditioned continuation (Phase 6), and an L2-normalised bi-encoder (Phase 7). Full fine-tuning, partial/layer-wise fine-tuning, LoRMA, a head-only probe and knowledge distillation are all implemented from scratch.

LoRMA is multiplicative where LoRA is additive: $h = I(BA)W_0x$ with $I(BA) = (\alpha/r)BA + I_d$, so the adapter begins as the identity. The codebase uses a fused `c_attn` of shape $(d, 3d)$, which the paper does not anticipate; one transform on the fused output would adapt Q, K and V jointly under a shared $(3d \times 3d)$ map, a different method. I slice and apply separate transforms to the Q and V thirds, leaving K untouched, verified by exact identity at initialisation and by merged weights reproducing the adapted forward pass to 8.3×10^{-7}.

4.3 Phase 2, Legal NER

I tag BIO over windows and reconstruct character offsets under `trim`. The error analysis drove everything after it: of 4,120 gold entities 45.2% were exact, but 27.0% were found with the *correct label and wrong span*, and 43.0% of predictions were spurious. Boundary errors are one-sided truncations of long citations, "R. Prakash v. State of Karnataka (2004) 9 SCC 27" predicted as "R", and 785 of 815 such gold spans round-trip exactly through the tokenizer, so this is neither a tokenizer limit nor a decoding bug: the BIO chain breaks mid-entity. Decoding alone, without retraining, moved fold-1 macro-F1 from 0.2541 to 0.3589, because strict orphan-I handling removes fragments (precision $0.223 \rightarrow 0.338$) while bridging gaps up to three tokens repairs chains (recall $0.291 \rightarrow 0.332$). That bridging helped monotonically was direct evidence a CRF should help, since it enforces valid transitions during inference instead of patching them afterwards. It did, by +0.18 cross-validated, the largest single gain in the project. I hard-mask the 44% of structurally impossible transitions rather than learning them from 56 documents.

4.4 Phase 3, Rhetorical role prediction

Sentences are separated by a reserved token and classified from that token's hidden state, with a domain marker so both domains train jointly; blocks are built predicted-region-first so no sentence is left unpredicted. The same CRF code worth +0.18 in Phase 2 produced a null result here (-0.002), and the contrast is the finding. BIO transitions are *hard* constraints and a CRF encodes impossibility exactly; rhetorical roles are merely autocorrelated, every transition is legal, and self-attention already propagates soft continuity. The conjecture this licenses is that a CRF helps in proportion to the fraction of its transition matrix that is structurally zero.

4.5 Phase 4, Court judgment prediction

With a median case of 2,669 tokens the only real design question is which tokens to keep, so I ran the three-way ablation, first 1024, last 1024, or first 512 plus last 512, holding everything else fixed. The tail wins at 0.7124 development macro-F1 against 0.5429 for the head (0.6997 against 0.5104 on the leaderboard). The head arm did not merely score lower, it collapsed to one class on 98% of development cases while its training loss kept falling, so the opening of a judgment carries no outcome signal to learn from. The head_tail arm is the more useful number, because spending half the budget on the head *costs* 0.025, head tokens are worth less than the tail tokens they displace, which is why I did not build a hierarchical encoder pooling chunks uniformly. Since training is 59/41 and evaluation balanced, evaluation also reports macro-F1 after subtracting $\tau \log p_{\text{train}}(y)$ from the logits; that correction is worth +0.023 and tracks the model's actual bias, +0.077 when it predicts the positive class 18% of the time and negative once it over-predicts.

4.6 Phase 5, Legal statute identification

The head is 100 independent sigmoids trained with binary cross-entropy; the interesting part is the decision rule. With the model held fixed, a 0.5 cutoff scores 0.2457, the best global threshold (0.15) scores 0.3572, and per-statute thresholds tuned on development data score 0.3849, a spread of +0.139 against 0.049 across all four adaptation techniques and architectures. A statute positive in few documents rarely reaches $p = 0.5$ even when ranked correctly, so a default cutoff drops exactly the tail macro-F1 averages over.

4.7 Phase 6, Abstractive summarization

Summaries have a median length of 749 tokens, so a summary alone nearly fills the window and no format puts document and summary in one context; I trained on chunk pairs, splitting each document and its reference summary proportionally. Before tuning anything I measured what the metric reads, and the answer reframed the phase. BERTScore uses `roberta-large`, whose tokenizer caps at 512 tokens: lead-512, lead-750, lead-1000 and the *entire document* all score exactly 0.8300, and appending 500 junk tokens past the cutoff leaves the score bit-identical. Generation length therefore has a hard optimum, fill the window, then stop, worth +0.032 from 256 to 512 tokens and exactly zero beyond, and the hierarchical machinery the brief emphasises can only serve the secondary ROUGE metrics, so I report that rather than building a hierarchy that cannot move the score.

4.8 Phase 7, Prior case retrieval

A case is represented as up to four 1024-token chunks over its first 4,096 tokens, and a pair is scored by maximum cosine similarity over chunk pairs; training samples one chunk from each side so training and inference perform the same operation. The 4,096-token budget is not arbitrary, a BM25 control peaks there at 0.2991 and falls monotonically to 0.0734 on full documents. Training uses InfoNCE over in-batch negatives.

4.9 Final submitted system

The checkpoints behind the submitted `prediction.json` files are: Phase 1 `arch2_prefix_3B`; Phase 2 `arm_b_docmask` with a CRF head, retrained on all 70 documents; Phase 3 `rr_decoder_plain`; Phase 4 `cjpe_tail` (tail-1024, last-token pooling, prior-corrected); Phase 5 a five-model ensemble with per-label thresholds; Phase 6 `summ_distil` (6-of-12-layer student, greedy with 4-gram blocking and a 480-token minimum); Phase 7 a four-model reciprocal-rank fusion. Each builder reads geometry and decision rule from the checkpoint rather than the command line (Section 5.1).

5 Experiments and Implementation Details

Table 4: Hyperparameters of the final model in each phase.

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Parameters (M)	110.3	110.3	110.3	110.3	110.4	68.0	110.3
Context length	1024	1024	1024	1024	1024	1024	4×1024
Peak LR	6e-4	3e-5	3e-5	3e-5	3e-5	3e-4	2e-5
Batch × accum.	32×8	8×1	4×1	8×1	8×1	4×1	16×1
Epochs / steps	5700 st	10 ep	10 ep	3 ep	4 ep	5 ep	5 ep
Dropout	0.0	0.1	0.1	0.1	0.1	0.1	0.0
Decoding / threshold	T0.8, p0.95	Viterbi	argmax	prior-corr.	per-label	greedy+norep4	max-sim

5.1 Local evaluation

Phase 1 uses a frozen 200-example set of random mid-document 512-word windows with the next 128 words as reference, because the real test prompts are arbitrary slices ending mid-sentence and a tidier set with clean

sentence breaks would be a different distribution. Phase 2 cross-validates across the official folds; Phases 3–7 use held-out splits.

Running the official-format checks before every upload caught missing and extra IDs, sentence-count mismatches, non-unique statute IDs, empty continuations and over-length summaries. One bug cost a score: in Phase 3 the submission builder constructed test blocks with default context settings while the checkpoint had been trained with a different geometry, costing 0.0095 (0.5014 → 0.4919). The symptom was visible before upload and I did not look: the majority label occupied 43.7% of test sentences against 32% on development. Every later builder reads geometry from the checkpoint, and I now compare the submitted label distribution against the model's own first.

Local and leaderboard scores agreed to 0.0034 in Phase 1, which validated the local evaluator, until it over-predicted one model by 0.0208. That prompted a contamination check: 42% of High Court evaluation windows have a near-duplicate in training at Jaccard ≥ 0.5, while the other three sources are clean (medians below 0.03). **A held-out set built by excluding exact hashes is not a held-out set**, and this is the most consequential methodological finding of the project; all Phase-1 numbers here use the 179-example filtered subset. Re-scoring moved the document-masking generation gap from +0.036 to +0.0153, so roughly 60% of the original headline was memorised near-duplicates.

5.2 Ablations and controlled comparisons

The Phase-1 ablation is document-boundary-aware attention: masking at `<|endoftext|>` boundaries so each packed document attends only within itself. I registered two predictions before running it, that the benefit would concentrate where boundary density is highest, and that it would grow at later positions within a block, precisely so a null could not be rationalised afterwards, and measured seed variance first at 0.005 validation loss by retraining the baseline under a fixed seed.

Both were refuted. Masking is *worse* by +0.0219 validation loss, four times seed variance and flat across 2,700 steps. The penalty does not track boundary density (Table 3): High Courts has twice the density of Acts and one-ninth the effect. It *grows* with position, and the pre-registered early-document confound is ruled out because masking helps at early positions (−0.010) and hurts at late ones (+0.050). Weighting Tribunal's +0.0466 by its 46.1% mixture share gives 0.0215, essentially the whole aggregate effect. This is not "masking hurts" but "masking hurts one source, and that source dominates the mixture." Masking assumes packed documents are independent, and for Tribunal roughly one adjacent pair in nine is the same matter filed twice, so cross-document attention is reading the same case. Neither factor alone suffices, Acts has relatedness but almost no boundaries, High Courts boundaries but nothing worth attending across, so the penalty behaves like density multiplied by relatedness, which makes this partly a near-duplicate problem wearing an architecture costume.

Three further controls: a hard-label arm for every distillation experiment, without which "distillation retains 94.5%" would have been attributed to the wrong mechanism; a head-only probe showing 0.18 of Phase 4's 0.7124 comes from adapting the encoder rather than from the pretrained representation; and a Phase-2 position-feature experiment that failed and explained itself, since APP and RESP have near-identical position distributions (medians 0.15 and 0.19).

6 Results

The generalisation gap differs per phase, explicably. Phase 5 is the cleanest: all seven submissions preserve their development ordering exactly, each offset by a near-constant −0.0156 (standard deviation 0.0015). That constant is the in-sample optimism of fitting 100 per-label thresholds on 10,181 documents, and it shifted the level without disturbing the order because every model paid the same amount. Phase 3 is the opposite: with 10 development documents, within-run epoch-to-epoch scores have a standard deviation of 0.021 while training loss falls monotonically, so best-of-twelve-epochs selection inflates by roughly $1.5\sigma \approx 0.03$ and varies per run, which is why a +0.028 development gain produced +0.000 on the leaderboard. Phase 7's larger offset (−0.038) is structural, its test pool holds 1,727 candidates against development's 1,023, and its rank-fusion submission, whose weights were chosen as the best of 324 configurations on 118 queries, kept only +0.0018 of a +0.0089 development gain.

Table 5: Per-phase variant comparison; local figures use each phase's own protocol.

Phase	Variant	Local	Codabench
1	decoder / encoder–decoder / windowed	0.8255 / 0.8311 / 0.8328	, / 0.8275 / 0.8308
	prefix-LM / decoder + doc. masking	**0.8358** / 0.8408	**0.8343** / 0.8339
2	full FT, prefix-LM / + CRF / LoRMA+ (0.28%)	0.2603 / 0.4438 / 0.4127	, / 0.5161 / 0.4839
	distillation (6 of 12) / **decoder + CRF**	0.4273 / **0.5044**	0.4366 / **0.5683**
3	no context / 24-sent. context / LoRMA / distilled	0.5617 / 0.5894 / 0.5248 / 0.5234	**0.5023** / 0.5014 /, / 0.4503
4	**full FT, tail-1024** / LoRMA (0.27%)	**0.7124** / 0.6836	**0.6997** / 0.6967
	distilled (61.5%) / head-1024 ablation	0.6911 / 0.5429	0.6954 / 0.5104
5	full FT / top-4-layer (25.7%) / LoRMA (0.34%)	0.4004 / 0.4004 / 0.3689	0.3849 / 0.3833 / 0.3557
	encdec / distilled / chunked max / **ensemble**	0.3512 / 0.4030 / 0.4114 / **0.4132**	0.3358 / 0.3888 / 0.3933 / **0.3978**
6	**distilled (61.5%)** / prefix-LM + LoRMA	**0.8273** / 0.8290	**0.8260** / 0.8251
	encdec / decoder top-4 / *lead-512 control*	0.8225 / 0.8084 / *0.8300*	0.8183 / 0.8074 /,
7	full FT / LoRMA (0.27%) / top-4-layer	0.3346 / 0.3202 / 0.3168	0.2947 / 0.2764 / 0.2812
	distilled / **RRF fusion** / *BM25 control*	0.3157 / **0.3435** / *0.2991*	0.2835 / **0.2965** /,

Three results decide which systems won. On Phase 1 the prefix-LM takes the matched comparison at 0.8358 despite five times less supervision per token processed. Downstream, no adaptation technique separated itself: Phase 4's top four submissions span 0.0079 on the leaderboard and comprise full fine-tuning at 100% of parameters, LoRMA at 0.27%, a distilled student at 61.5% and a different input window, while Phase 7's four span 0.018. What did separate was representation and decision rule, the 0.189 position gap in Phase 4, the +0.139 threshold gap in Phase 5, the +0.061 free gain from chunked max-similarity in Phase 7, and the +0.040 decoding gap in Phase 6.

Per-class results confirm that macro-F1 hides dead labels. Phase 2 splits between form-recognisable and role-dependent entities: DATE 0.9332, COURT 0.7296 and STAT 0.6376 against WIT 0.0156, RESP 0.0948 and APP 0.1030, with R.COUNSEL predicted as A.COUNSEL 55 times. Phase 5's test micro-F1 is 0.49 against macro 0.39, and Phase 3's RatioOfTheDecision absorbs 399 false positives as the default for anything reasoning-shaped.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phase 1's dominant failure is degeneration: the unmasked arm has a distinct-4-gram ratio of 0.7825 against 0.9797 for the masked arm, and falls short of the 128-word target five times as often. Both are what BERTScore punishes, repetition destroys precision, short output destroys recall, so the deficit is degeneracy rather than subtle quality. Phase 2 truncates long citations (27% of gold entities) and never solves role assignment; Phase 3 over-predicts the majority rhetorical role; Phase 4's head arm collapses to one class; Phase 5 misses rare statutes under any global threshold; and Phase 7 fails on queries whose relevance is not recoverable from the first 4,096 tokens. Phase 6 produces fluent prose that does not match the reference's wording: the encoder–decoder ranks third on BERTScore while ranking *last* on ROUGE-L and BLEU-4 with the cleanest generation statistics of any model, which is what a span-reconstruction objective should give.

7.2 What the metrics do not measure

BERTScore compares contextual embeddings over at most 512 tokens, which here largely measures whether two passages concern the same kind of matter. With 86.1% token-type overlap between references and sources, printing the first 512 tokens verbatim scores 0.8300 and no abstractive model I built beat it, a statement about the metric as much as the model, since a firm would care whether the holding and the statute survive into the summary, which BERTScore does not ask. Strict span-F1 rewards offset perfection over entity typing, and Phase-7 gold sets are incomplete at 8.25 labelled candidates out of 1,023, so an unlabelled but genuinely relevant retrieval is punished like an irrelevant one.

7.3 Limitations and what I would do next

Every Phase-1 architecture comparison is matched on tokens read rather than supervised positions, so the prefix-LM wins with a fifth of the decoder's supervision and the encoder–decoder with a seventh; matching supervision would need five to seven times the compute. Phases 2 and 3 cannot separate "causal attention suits this task" from "this checkpoint is simply better", because the control on a plain baseline decoder was descoped for time, 20 minutes of GPU, and the first thing I would run. Next is re-running the Phase-1 ablation on a MinHash-deduplicated corpus, which should make the Tribunal penalty disappear if Section 5.2 is right, and a training-set-size sweep on Phase 4's distillation. No human legal evaluation was performed, and phases were trained independently rather than pipelined, so no error propagation was measured.

7.4 Surprising findings

Eight pre-registered predictions were refuted, and the failures carried more information than the confirmations. Document masking, standard practice at scale, made validation loss worse here, because its independence assumption is empirically false for one source. Bidirectional encoding was predicted to beat causal attention on span tagging and on rhetorical roles and lost both, by 0.052 and 0.110. Hard negatives mined from BM25 made retrieval slightly worse, most plausibly because the top-ranked non-gold candidates are precisely the unlabelled positives.

The most useful surprise is that a decision rule can invert a model comparison. In Phase 6, under the training loop's fixed greedy decoding the encoder–decoder appears to beat the prefix-LM by 0.012; under each model's own best decoding the prefix-LM wins by 0.008, because greedy decoding sends it into a 0.806-repetition loop that one 4-gram-blocking flag removes. In Phase 5 max-pooling over chunks appears 0.07 better than mean-pooling at a 0.5 cutoff and ties it once thresholds are tuned, and an ensemble evaluated at 0.5 scores worse than every one of its members. In three of four phases the default decision rule would have given the wrong answer.

Distillation produced a five-point series rather than a number: the KL term contributed +0.0036, +0.0050, +0.0354, +0.0083 and +0.0133 across Phases 2–7, against truncation costs of 0.071, 0.057, 0.006, 0.002 and 0.032 and training sets from 70 to 43,000 examples. The working account is that the soft target helps in proportion to what halving the depth costs, and only with enough data to exploit it, Phase 4 is the only phase with both and the only large effect. It predicted Phases 5 and 6 and mispredicted Phase 7, so I report it as a working account, not an established one. In three phases the half-size student matched or beat its teacher.

8 Conclusion

The delivered system is one 110M-parameter legal language model trained from scratch on 3.5B tokens under a strict parameter and data-source lock, with six task-specific descendants running on premises. The two choices that moved metrics were not the ones the brief emphasises. The first is which part of a document the model reads: 0.189 macro-F1 separates the last 1024 tokens of a judgment from the first in Phase 4, and the optimum reverses between tasks: statute identification wants the head, outcome prediction the tail. The second is the rule converting scores into predictions, worth +0.139 in Phase 5 and +0.040 in Phase 6 and capable of inverting a ranking. The adaptation technique did *not* move metrics, and capacity was never the binding constraint, a 110M model memorises 90 documents to 0.996 macro-F1 while generalising at 0.56. What binds is labelled data, how much of each document the window admits, and whether the evaluation set can rank models at the resolution of the effects being chased.

A sceptical reader should not trust Phase 3's 10-document development set; the Phase-2 and Phase-3 architecture comparisons, confounded with checkpoint quality; or the summarizer, which does not beat printing the document's first 512 tokens, on a metric that cannot see past them.

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? *Answer:* Claude (Anthropic).

Q2. For each row, mark Yes or No and state the purpose.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Clarifying LoRMA, CRF and distillation formulations.
Ideation / brainstorming at a high level	Yes	Candidate experiment directions; selection was mine.
Debugging small ancillary snippets	Yes	Shape mismatches, argparse and masking bugs.
Plotting / visualization code	Yes	Position-delta and training-curve plots.
Grammar / writing style of the report	Yes	Phrasing and copy-editing.
Report structure / section wording	Yes	Section ordering and headings.
Designing the model architecture	Yes	Options discussed; final choices and verification mine.
Hyperparameter search / tuning	Yes	Suggested ranges; runs and selection mine.
Analysis of results / conclusions	Yes	Drafting and sanity-checking interpretations.
Writing the core training / inference code	Yes	Getting rough outline of training scripts
Other (specify)	No	

Q3. Did you refer to sources other than lecture slides and an LLM? *Answer:* Yes, the papers listed below, the official competition evaluation scripts, and the PyTorch, `tokenizers`, `bert-score` and `rouge-score` documentation.

Q4. Apart from any permitted LLM use, what was your contribution? *Answer:* I designed and verified the Phase-1 foundation model myself, settling on a 12-layer, 768-dimensional pre-norm decoder with tied embeddings after working out that tying frees roughly 24.6M parameters, about two extra layers against a fixed budget. Before committing GPU time I insisted on a verification suite: causality and padding isolation tests, a manual-versus-FlashAttention agreement check, an initial-loss check against $\ln 32000$, and a kill-and-resume test. I chose the 32k vocabulary after measuring fertility at four sizes, and tested explicit digit splitting expecting it to help; it cost 7.6% fertility and a token dump showed byte-level BPE already splits digits, so I dropped it. For the document-masking ablation I registered two predictions in writing beforehand, so that a null could not be reinterpreted afterwards, and measured seed variance first by retraining the baseline under a fixed seed. Both predictions were refuted, and tracing why led me to the near-duplicate structure of the Tribunal source, which explains the result better than the architecture does. That investigation made me check the held-out set for contamination; finding near-duplicates in training for 42% of one source's evaluation windows forced me to revise my own headline result down by about 60%. In Phase 2 my error analysis showed 27% of gold entities were found with the correct label and the wrong span, which made the CRF an evidence-backed choice rather than a guess. In Phase 4 I ran the position ablation showing the outcome signal sits at the end of a judgment, and in Phases 5 and 6 I measured the decision rule, thresholds and decoding, instead of assuming it was a detail. Two things failed. Position features in Phase 2 did nothing, and working out why was the lesson: APP and RESP have nearly identical position distributions, so a feature giving both the same value cannot separate them. BitFit in Phase 4 proved inapplicable to my own architecture, since I had built the models without bias terms, so the arm trained 1,538 parameters and became a head-only probe; separately, retraining on all data before submitting cost me 0.040 on the Phase-3 leaderboard, which taught me that with a memorisable dataset the held-out split is worth more as a stopping signal than as extra training data.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv:1503.02531*, 2015.

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *ICLR*, 2022.

[4] A. Sharma et al., "LoRMA: Low-rank multiplicative adaptation for large language models," *Findings of ACL*, 2025.

[5] E. Ben Zaken, S. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language models," *ACL*, 2022.

[6] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv:1910.01108*, 2019.

[7] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *JMLR*, vol. 21, 2020.

[8] J. Hoffmann et al., "Training compute-optimal large language models," *NeurIPS*, 2022.

[9] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *ICLR*, 2020.

[10] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, 2009.

[11] Y. Zhao et al., "Analysing the impact of sequence composition on language model pre-training," *ACL*, 2024.

[12] V. Malik et al., "ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation," *ACL*, 2021.

[13] P. Kalamkar et al., "Named entity recognition in Indian court judgments," *Natural Legal Language Processing Workshop*, 2022.

[14] P. Paul, P. Goyal, and S. Ghosh, "LeSICiN: A heterogeneous graph based approach for automatic legal statute identification from Indian legal documents," *AAAI*, 2022.